Falco-lite

Агент runtime-захисту Linux на eBPF

Бачить атаку на рівні ядра і зупиняє її за мілісекунди: reverse shell, запуск із /tmp, читання /etc/shadow, підвищення привілеїв, майнери.

```
root@web-01
$ sudo ./falco-lite --config rules.yaml
[+] eBPF: завантажено 5 програм (execve, connect, openat, setuid, ptrace)
[+] ring buffer підключено, ядро 6.8, CO-RE
[+] правила: 5 із rules.yaml
[+] спостерігаю…
```

Микита Вельбовець, Томас Іванаускас, Назар Дяченко

Команда та проєкт

Falco-lite — демон на C, який через eBPF спостерігає за системними викликами в ядрі Linux і в реальному часі виявляє атаки: reverse shell, запуск із /tmp, читання /etc/shadow, підвищення привілеїв, майнери. Правила описуються у YAML-конфігу; при спрацюванні агент може сам убити процес. Стек: C11, libbpf, libyaml; Linux 5.8+ на x86-64.

Модуль	Учасник	Відповідальність
eBPF-програми	Микита Вельбовець	Хуки на execve, connect, openat, setuid, ptrace; передача подій у userspace через ring buffer
Рушій правил	Томас Іванаускас	Парсер YAML-конфігу, зіставлення подій із правилами, дерево процесів: хто кого запустив
Реакція та вивід	Назар Дяченко	kill і alert, JSON-логи, відправка у Wazuh або syslog, TUI на ncurses; CI та тести

Мета проєкту

Побачити атаку в момент виконання, на рівні системних викликів, і зупинити її автоматично, без участі людини.

Класичний антивірус дивиться на файли та сигнатури. Атака на сервері — це поведінка процесів: хто кого запустив, куди підключився і що прочитав.

Бачити

Кожен execve, connect, openat, setuid і ptrace — просто з ядра через eBPF, без модулів ядра і патчів.

Розпізнавати

Правила у YAML: хто запустив, звідки, куди підключається, що читає. Контекст дає дерево процесів.

Реагувати

SIGKILL за мілісекунди після події; алерт у JSON, syslog або Wazuh — без участі адміністратора.

Чому eBPF

Підхід	Що бачить	Обмеження
auditd	Події ядра, постфактум	Без дерева процесів і без реакції
LD_PRELOAD	Виклики libc у динамічних бінарниках	Статичні бінарники та прямі системні виклики невидимі
ptrace, strace	Усі системні виклики одного процесу	Зупиняє процес на кожному виклику — надто повільно для продакшену
Модуль ядра	Усе	Помилка в коді — паніка ядра; жорстка прив’язка до версії ядра
eBPF	Усе, в реальному часі, з кожного процесу	Код перевіряє верифікатор ядра; CO-RE — один бінарник для різних версій ядра

Атаки, які ловить агент

Атака	Приклад	Що бачить агент	Реакція
Reverse shell	bash -i >& /dev/tcp/...	Shell викликає connect() назовні, а його батько — веб-сервер або cron	SIGKILL
Запуск із /tmp	/tmp/.x, /dev/shm/.x	execve з файлом у /tmp, /dev/shm або /var/tmp — так стартують щойно завантажені payload-и	SIGKILL
Читання /etc/shadow	cat /etc/shadow	openat("/etc/shadow") процесом не з allow-list (sshd, sudo, passwd, login); невдала спроба теж	Алерт
Підвищення привілеїв	setuid(0), ptrace	setuid(0) без sudo або su в дереві процесів; ptrace до чужого процесу	SIGKILL
Криптомайнер	xmrig -o pool:3333	execve відомих майнерів або connect() на порти stratum-пулів (3333, 4444, 14444)	SIGKILL

Де застосовується

Веб-сервер або VPS

Web shell у php-fpm чи node переходить у reverse shell. Агент убиває shell раніше, ніж атакувальник побачить промпт.

CI-раннери та build-агенти

Тут постійно виконується чужий код: supply-chain атака або майнер у pipeline — типовий сценарій.

Контейнери та Kubernetes

Ядро спільне для всіх контейнерів, тож один агент на хості бачить процеси кожного з них.

SOC та SIEM

Falco-lite працює як сенсор: події йдуть у syslog або Wazuh, а кореляція та дашборди — на боці SIEM.

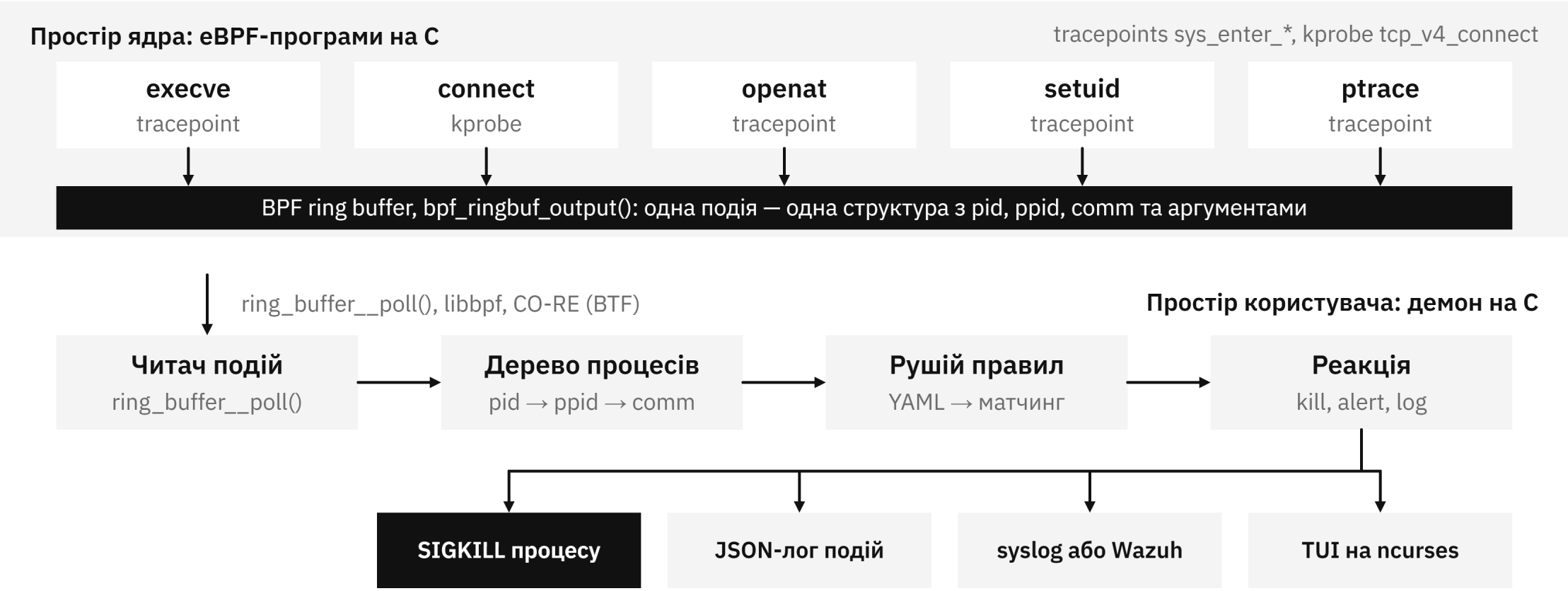
Honeypot і навчальні лабораторії

JSON-лог дій атакувальника — матеріал для розбору інцидентів, CTF і тренувань SOC.

Домашній сервер

Легкий агент замість комерційного EDR: правила у YAML, жодних хмарних залежностей.

Архітектура



Модулі, стек і правила

rules.yaml

```yaml
- name: reverse_shell
  when:
    syscall: connect
    proc.comm: [bash, sh, dash, zsh]
    proc.ancestor: [php-fpm, nginx, cron]
  action: kill

- name: shadow_read
  when:
    syscall: openat
    file.path: /etc/shadow
    proc.comm.not: [sshd, sudo, passwd, login]
  action: alert
```

eBPF-програми, ядро

C, зібраний clang для цілі bpf; libbpf з CO-RE (BTF); skeleton через bpftool; BPF ring buffer.

Рушій правил, userspace

libyaml читає правила у масив; дерево процесів у хеш-таблиці (pid, ppid, comm, exe); матчинг кожної події.

Реакція та вивід, userspace

kill(pid, SIGKILL); JSON-лог; syslog або Wazuh; TUI на ncurses; CI на GitHub Actions і тести.

```
$ make && sudo ./falco-lite -c rules.yaml
```

Linux 5.8+ (BPF ring buffer, CAP_BPF), один статичний бінарник, без модулів ядра


Linux


C11


Clang / LLVM


YAML


GitHub Actions


Docker


Kubernetes

Демонстрація: атака і реакція

Сценарій на тестовому сервері

Reverse shell
Shell, запущений веб-сервером, підключається назовні

Запуск із /tmp
Завантажений файл виконується з тимчасової теки

Читання /etc/shadow
Процес поза allow-list відкриває файл паролів

root@web-01, falco-lite — **спостерігаю**

```
[+] 5 eBPF-програм, ring buffer, 5 правил
17.102 EXEC     pid=4213 bash ← php-fpm(1337)
17.104 CONNECT  pid=4213 → 10.0.0.66:4444/tcp
17.105 ALERT    reverse_shell  critical
17.106 ACTION   kill(4213, SIGKILL)  4 ms
41.890 EXEC     pid=4305 /tmp/.x ← bash(4210)
41.890 ALERT    exec_from_tmp  high
41.891 ACTION   kill(4305, SIGKILL)  1 ms
58.410 OPENAT   pid=4310 cat /etc/shadow
58.410 ALERT    shadow_read  high
58.411 ACTION   alert → json, syslog, wazuh
підсумок: 3 події, 2 процеси зупинено, 1 алерт
```

execve → connect → правило reverse_shell → kill(4213, SIGKILL): від системного виклику до реакції минають мілісекунди.

Дякуємо за увагу.